

CivicPulse AI

Team Development & Integration Standards

PURPOSE

This document serves as the common development rulebook for our 5-member engineering team. We are developing our modules independently, but they must seamlessly integrate into a unified application. Adherence to these standards ensures minimal integration friction.

"Independent modules, shared contracts."

MODULES OVERVIEW:

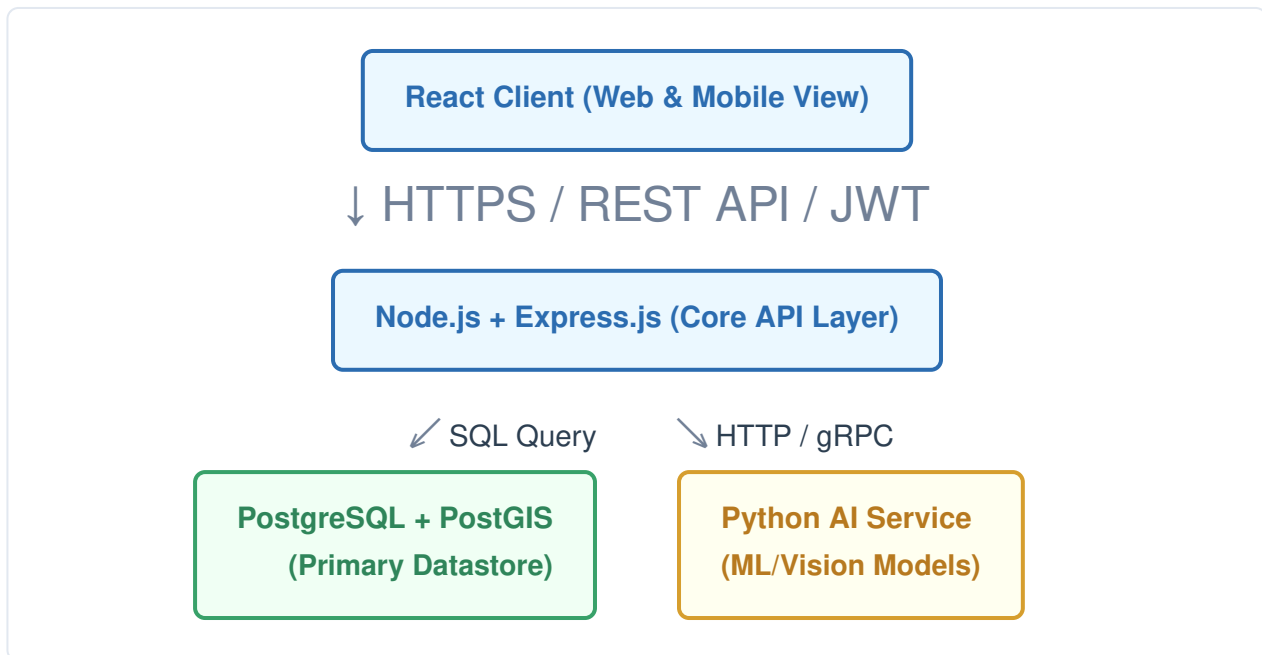
1. Issue Reporting
2. AI Issue Analysis
3. Duplicate Detection + Community Verification
4. Priority + Authority Management
5. Civic Dashboard + Analytics Map

TECH STACK:

React + Tailwind CSS | Node.js + Express.js | PostgreSQL + PostGIS | Python AI/ML | REST APIs | JWT | Git/GitHub | Docker

1. SYSTEM ARCHITECTURE

Our architecture follows a microservice-inspired modular monolith design. The React client communicates with a central Node.js API, which manages the primary PostgreSQL/PostGIS database and orchestrates calls to the Python AI microservice.



Module Connection Flow:

The five modules act as distinct domain boundaries within the same application ecosystem. The React frontend consumes APIs specific to each module's domain. The Node backend routes Issue Creation logic first, triggers the Python AI Service asynchronously for analysis, executes duplicate detection queries in PostGIS, and hands over to the Priority/Authority engine for assignment.

2. SHARED DEVELOPMENT CONVENTIONS

To guarantee smooth integration, every developer MUST strictly adhere to the following conventions:

- **Naming Conventions:** Use `snake_case` for ALL database tables, columns, and API request/response JSON fields. Use `PascalCase` for React components and `camelCase` for internal JavaScript/TypeScript variables.
- **Authentication:** Centralized User entity. Use JWT (JSON Web Tokens) attached in the `Authorization: Bearer <token>` header. *Do not implement secondary auth systems.*
- **Central Issue Entity:** The `Issue` is the atomic unit of the application. All modules orbit this entity.
- **Unique Issue ID:** A standardized, human-readable ID format must be used across all modules (e.g., `CP-YYYY-XXXXXX`). Do not use bare integers or raw UUIDs in API responses for issue references.
- **Status & Categories:** Must use exactly the defined Enums (see Section 5).
- **Geospatial Data:** Store coordinates as `latitude` and `longitude` in JSON payloads. In PostgreSQL, store them as a PostGIS `geometry(Point, 4326)`.
- **Media Structure:** Store media URLs as an array of strings in JSON, referencing an S3-compatible cloud storage bucket or a standardized static file path.
- **Timestamps:** All timestamps must use ISO 8601 format (e.g., `2026-08-18T21:54:33Z`) in API responses and UTC timezone in the database.
- **Environment Variables:** Use standardized prefixes. Database vars: `DB_HOST`, `DB_USER...`, Auth: `JWT_SECRET`. A shared `.env.example` will be maintained.

MANDATORY RULE

If a convention is missing from this document, it must be discussed and appended here before implementation.

3. CENTRAL ISSUE CONTRACT

The **Issue** object is the central nervous system of CivicPulse. Different modules populate different fields, but the structure remains unified.

Example Shared Contract:

```
{
  "issue_id": "CP-2026-000123",
  "user_id": "USR-001",
  "description": "Massive pothole causing traffic slowdowns.",
  "category": "ROAD",
  "subcategory": "POTHOLE",
  "latitude": 18.5204,
  "longitude": 73.8567,
  "status": "SUBMITTED"
}
```

Data Ownership Breakdown:

- **Issue Reporting Module:** Generates **issue_id**, **user_id**, initial **description**, raw **latitude** / **longitude**, and sets initial **status** to **SUBMITTED**.
- **AI Module:** Appends or updates **category**, **subcategory**, and generates an AI confidence score (stored in a related table).
- **Duplicate Detection:** Updates the **status** to **DUPLICATE** if matched, and establishes a parent-child relationship between issues.
- **Priority + Authority Module:** Appends priority scores, updates **status** to **ASSIGNED**, and links authority user IDs.

4. DATABASE DESIGN

Do NOT create one massive "Issues" table containing 100 columns. We will use a normalized schema to respect module boundaries while maintaining relational integrity.

Proposed Shared Structure (Core Tables):

- **users** : Centralized identity (Citizens and Authorities).
- **issues** : The core entity. Contains ONLY the ID, location, author, timestamps, and current status/category.
- **issue_media** : 1-to-Many mapping of issue images/videos.
- **issue_status_history** : Audit trail of all status changes.
- **issue_ai_analysis** : Owned by AI Module. Stores confidence scores, detected objects, and automated text summaries.
- **issue_duplicates** : Owned by Duplicate Detection. Maps **issue_id** to **parent_issue_id**.
- **issue_assignments** : Owned by Priority/Authority. Maps issues to departments/workers.

Entity Relationship Diagram (Simplified):

```
[users] 1 -- * [issues]
[issues] 1 -- * [issue_media]
[issues] 1 -- * [issue_status_history]
[issues] 1 -- 1 [issue_ai_analysis]
[issues] 1 -- * [issue_duplicates]
[issues] 1 -- * [issue_assignments]
```

Data Ownership: A module is the "owner" of its designated tables. For example, the Reporting module should not write directly to **issue_ai_analysis**, and the AI module should not directly modify **issue_assignments**.

5. COMMON ENUMS

Enums ensure data consistency across the frontend dropdowns, backend logic, and database schemas. These values must remain IDENTICAL across all environments.

Status Values

- SUBMITTED
- UNDER_REVIEW
- VERIFIED
- ASSIGNED
- IN_PROGRESS
- RESOLVED
- CLOSED
- DUPLICATE
- REJECTED

Category Values

- ROAD
- WASTE
- WATER
- DRAINAGE
- STREETLIGHT
- TRAFFIC
- PUBLIC_INFRASTRUCTURE
- OTHER

6. API CONTRACT

Every REST endpoint across all Node.js and Python services MUST adhere to a unified response wrapper.

Success Format

```
{
  "success": true,
  "message": "Operation completed successfully.",
  "data": { ... }
}
```

Error Format

```
{
  "success": false,
  "message": "Human-readable error description.",
  "error_code": "ERR_DUPLICATE_ENTRY"
}
```

Proposed Endpoint Ownership:

Note: These are PROPOSED and must be finalized by the respective module owners during the first sprint.

- **Issue Reporting:**

- POST /api/issues

- GET /api/issues/:id

- GET /api/issues/my

- **AI Module:**

- POST /api/ai/analyze

- **Duplicate Detection:**

- POST /api/duplicates/check

- **Priority/Authority:**

- POST /api/issues/:id/priority

- POST /api/issues/:id/assign

- **Dashboard:**

- GET /api/dashboard/stats

- GET /api/dashboard/heatmap

7. MODULE INTEGRATION

Modules do not operate in a vacuum. A typical issue lifecycle passes through multiple modules in a pipeline.

Issue Reporting → AI Analysis → Duplicate Detection →

Priority Setup → Authority Assignment → Dashboard Vis

Data Exchange Rule: Use the SAME `issue_id` everywhere. When Module A calls Module B, it passes the `issue_id`, allowing Module B to pull context from the database if necessary, rather than passing massive payloads over HTTP.

Concise Integration Contracts:

- **AI Analysis Request:** `{ "issue_id": "CP-2026-123", "image_url": "...", "category": "ROAD", "confidence": 0.92 }`
- **Duplicate Check Request:** `{ "issue_id": "CP-2026-123", "lat": 18.52, "lng": 73.85 }`
- **Duplicate Response:** `{ "is_duplicate": true, "parent_id": "CP-2026-010" }`

8. GIT + PROJECT STRUCTURE

We use a monorepo setup to maintain visibility across the stack, but directories remain strictly separated by concern.

```
civicpulse-ai/  
├─ client/           # React frontend  
├─ server/           # Node.js backend  
├─ ai/               # Python AI service  
├─ docs/             # Shared architecture & API docs  
├─ docker-compose.yml # Local environment setup  
├─ .env.example       # Shared environment variables template  
└─ README.md         # Bootstrapping instructions
```

Git Workflow (Trunk-Based with Feature Branches):

Branching Strategy: `main` → `develop` → `feature/<module-name>-<task>`

- **No Direct Pushes:** Pushing directly to `main` or `develop` is strictly forbidden.

- **Pull Requests (PRs):** All code merges via PRs.
- **Code Review:** At least ONE other team member must review integration-level PRs.
- **Meaningful Commits:** Write descriptive commit messages (e.g., `feat(ai): integrate pothole detection model`).
- **Regular Merging:** Merge `develop` into your feature branch daily to prevent massive merge conflicts later.

9. WHAT MUST NOT BE DONE

The following are integration-breaking mistakes. Committing these errors will block the entire team.

CRITICAL ANTI-PATTERNS:

- Creating separate authentication systems for different modules.
- Duplicating User or Issue entities in the database.
- Using different, non-standard Issue IDs.
- Using variations of status/category names (e.g., `UNDER_REVIEW` vs `Reviewing`).
- Using different location formats (e.g., `[lat, lng]` vs `{x, y}`).
- Creating duplicate endpoints that serve the same purpose.
- Directly modifying another module's database tables without using their API.
- Hardcoding local URLs (`localhost:3000`) or secrets in the codebase.
- Committing `.env` files or production secrets to Git.
- Making independent database schema changes without team approval.
- Changing shared API contracts silently without notifying the frontend developer.
- **Waiting until the very end of the project to attempt integration.**

10. INTEGRATION STRATEGY

We will NOT develop in silos for six weeks and hope things connect on the final day. Integration is continuous.

1. **Phase 1: Shared Contracts (Week 1).** Finalize all database schemas, JSON payloads, and API endpoints before writing core logic.
2. **Phase 2: Mock Implementations (Week 2).** Develop modules using static mocked data that adheres to the contract.
3. **Phase 3: Early API Integration (Week 3).** Connect the React frontend to the Node API, and the Node API to the Python service using basic/dummy processing logic.
4. **Phase 4: Regular Integration Testing (Weekly).** Pull latest code, run `docker - compose up`, and ensure the end-to-end Issue Creation flow works locally.
5. **Phase 5: Final Polish.** Refine complex logic now that the pipelines are proven to work.

11. RESPONSIBILITY MATRIX

Clear boundaries prevent overlapping work and define exactly who to consult when integrating.

Module	Core Responsibilities	Dependencies
Issue Reporting	Issue creation, media uploads, initial location, ID generation, JWT auth validation.	None (Initiates Flow)
AI Analysis	Image analysis, categorization, confidence scoring, NLP description summaries.	Needs image URL & text from <i>Issue Reporting</i> .
Duplicate Detection	Geospatial queries, similarity matching, community upvoting logic.	Needs PostGIS location data & category from <i>AI Analysis</i> .

Module	Core Responsibilities	Dependencies
Priority/ Authority	Scoring logic, authority worker assignments, notification triggers.	Needs deduplicated issue status & category.
Dashboard	Analytics maps, heatmaps, resolution time metrics, administrative visualizations.	Depends on ALL data being correctly formatted.

12. SHARED DOCUMENTATION

Documentation is a shared responsibility. The `/docs` folder in the monorepo must be kept up to date.

- `/docs/ARCHITECTURE.md` : High-level system diagrams and deployment steps.
- `/docs/API_CONTRACT.md` : The single source of truth for all Request/Response JSON payloads.
- `/docs/DATABASE_SCHEMA.md` : Master copy of the PostgreSQL table definitions.
- `/docs/DATA_DICTIONARY.md` : Explanations of complex fields (e.g., how Priority Score is calculated).
- `/docs/ENUMS.md` : Master list of allowed statuses and categories.
- `/docs/INTEGRATION_RULES.md` : A summarized version of this PDF for quick reference.

13. FINAL CHECKLIST

Every team member must review and agree on these items BEFORE serious feature development begins:

- ☐ Architecture approved
- ☐ Database schema finalized and normalized

- ❑ Auth mechanism (JWT) configured
- ❑ Issue entity structure locked
- ❑ ID generation strategy agreed upon
- ❑ Enums (Categories/Status) strictly defined
- ❑ Location (PostGIS/Coords) format finalized
- ❑ Media storage approach decided
- ❑ API request/response contracts documented
- ❑ Standard error format adopted
- ❑ API endpoint ownership mapped out
- ❑ AI integration contract established
- ❑ Duplicate detection parameters set
- ❑ Git workflow understood by all
- ❑ Environment variables synced via `.env.example`
- ❑ Docker-compose running locally for everyone
- ❑ Documentation placeholders created

14. FINAL TEAM RULES

Keep these ten rules in mind daily:

1. Commit code daily to your branches; merge `develop` frequently.
2. Never change a shared API contract without a team discussion.
3. Respect module boundaries: interact via APIs, not direct unauthorized database hacks.
4. If your module breaks the build, fixing it becomes your highest priority.
5. Write code assuming the frontend expects exactly what the API contract dictates.
6. Test your features against the shared Docker environment, not just your local standalone script.
7. Log errors gracefully using the standard error JSON format.

8. Keep secrets strictly in your local `.env` file.
9. Over-communicate on Slack/Discord when changing data structures.
10. **And most importantly...**

"We can develop independently, but we cannot define shared things independently."